Airflow 04 Xcoms

Source originale : https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/xcoms.html

↑↓

Navigate

■

Select

Esc

Close

XComs

XComs (short for “cross-communications”) are a mechanism that let

Tasks

talk to each other, as by default Tasks are entirely isolated and may be running on entirely different machines.

An XCom is identified by a

key

(essentially its name), as well as the

task_id

and

dag_id

it came from. They can have any serializable value (including objects that are decorated with

@dataclass

or

@attr.define

, see

TaskFlow arguments

:), but they are only designed for small amounts of data; do not use them to pass around large values, like dataframes.

XCom operations should be performed through the Task Context using

get_current_context()

. Directly updating using XCom database model is not possible.

XComs are explicitly “pushed” and “pulled” to/from their storage using the

xcom_push

and

xcom_pull

methods on Task Instances.

To push a value within a task called

“task-1”

that will be used by another task:

pushes data in any_serializable_value into XCom with key "identifier as string"

task_instance

.

xcom_push

(

key

=

"identifier as a string"

,

value

=

any_serializable_value

)

To pull the value that was pushed in the code above in a different task:

pulls the XCom variable with key "identifier as string" that was pushed from within task-1

task_instance

.

xcom_pull

(

key

=

"identifier as string"

,

task_ids

=

"task-1"

)

Many operators will auto-push their results into an XCom key called

return_value

if the

do_xcom_push

argument is set to

True

(as it is by default), and

@task

functions do this as well.

xcom_pull

defaults to using

return_value

as key if no key is passed to it, meaning it’s possible to write code like this:

Pulls the return_value XCOM from "pushing_task"

value

=

task_instance

.

xcom_pull

(

task_ids

=

'pushing_task'

)

If you need to pull a value from a specific DAG run (for example, a DAG run triggered by

TriggerDagRunOperator

), specify both

dag_id

and

run_id

explicitly:

trigger_run_id

=

task_instance

.

xcom_pull

(

task_ids

=

"trigger_child"

,

key

=

"trigger_run_id"

)

child_value

=

task_instance

.

xcom_pull

(

task_ids

=

"child_task"

,

dag_id

=

"child_dag"

,

run_id

=

trigger_run_id

,

)

The return_value key (default key with which XComs are pushed) is defined as a constant XCOM_RETURN_KEY in the

BaseXCom

class and can be accessed as BaseXCom.XCOM_RETURN_KEY.

You can also use XComs in

templates

:

SELECT

*

FROM

{{

task_instance

.

xcom_pull

(

task_ids

=

'foo'

,

key

=

'table_name'

)

}}

Note

xcom_pull()

without a

task_ids

argument pulls only from the current task. In Airflow 2, the same call would search all tasks and return the most recent value. Always specify

task_ids

explicitly when pulling from other tasks.

XComs are a relative of

Variables

, with the main difference being that XComs are per-task-instance and designed for communication within a Dag run, while Variables are global and designed for overall configuration and value sharing.

If you want to push multiple XComs at once you can set

do_xcom_push

and

multiple_outputs

arguments to

True

, and then return a dictionary of values.

An example of pushing multiple XComs and pulling them individually:

A task returning a dictionary

@task

(

do_xcom_push

=

True

,
multiple_outputs
=
True
)
def
push_multiple
(
**
context
):
return
{
"key1"
:
"value1"
,
"key2"
:
"value2"
}
@task
def
xcom_pull_with_multiple_outputs
(
**
context
):
Pulling a specific key from the multiple outputs
key1
=
context
[
"ti"
]
.

```
xcom_pull
(
task_ids
=
"push_multiple"
,
key
=
"key1"
)
# to pull key1
key2
=
context
[
"ti"
]
.
xcom_pull
(
task_ids
=
"push_multiple"
,
key
=
"key2"
)
# to pull key2
# Pulling entire XCom data from push_multiple task
data
=
context
[
"ti"
]
```

.

xcom_pull

(

task_ids

=

"push_multiple"

,

key

=

"return_value"

)

Note

If the first task was not successful then on every retry task XComs will be cleared to make the task run idempotent. XComs therefore can’t be used to persist state across task retries or

Sensors

poke.

Object Storage XCom Backend

The default XCom backend, BaseXCom, stores XComs in the Airflow database, which works well for small values but can cause issues with large values or a high volume of XComs. To overcome this limitation, object storage is recommended for efficiently handling larger data. For a detailed overview, refer to the

documentation

.

Custom XCom Backends

The XCom system has interchangeable backends, and you can set which backend is being used via the

xcom_backend

configuration option.

If you want to implement your own backend, you should subclass

BaseXCom

, and override the

serialize_value

and

deserialize_value

methods.

You can override the

purge

method in the

BaseXCom

class to have control over purging the XCom data from the custom backend. This will be called as part of

delete

.

Verifying Custom XCom Backend usage in Containers

Depending on where Airflow is deployed i.e., local, Docker, K8s, etc. it can be useful to be assured that a custom XCom backend is actually being initialized. For example, the complexity of the container environment can make it more difficult to determine if your backend is being loaded correctly during container deployment. Luckily the following guidance can be used to assist you in building confidence in your custom XCom implementation.

If you can exec into a terminal in an Airflow container, you can then print out the actual XCom class that is being used:

from

airflow.sdk.execution_time.xcom

import

XCom

print

(

XCom

.

__name__

)

Was this entry helpful?